

🔗 本章目标

诚实认识 C++ 的反射现状:标准至今没有运行时反射,动态特性由 RTTI(`typeid`、`dynamic_cast`)、类型萃取(编译期反射)与宏/代码生成三块拼成。掌握这三块工具的能力边界,知道生产环境用哪些库补齐反射,并理解多态、序列化、日志等场景的真实解法。

📦 前置知识

熟悉虚函数与多态(第6章)、模板与特化(第10章)、强制转换的初步概念。本章代码基于 C++17;RTTI 默认开启,若用 `-fno-rtti` 编译,`typeid` 与 `dynamic_cast` 不可用。

§11.1 没有反射的 C++	§11.2 <code>typeid</code> 与 <code>type_info</code>	§11.3 <code>dynamic_cast</code> 安全转型
§11.4 编译期类型查询	§11.5 宏与代码生成	§11.6 生产环境的选择

§11.1 先说实话:C++ 没有标准反射

Java 的 `getClass()`、C# 的 `typeof` 都能在运行时拿到类型的完整信息:字段列表、方法表、注解,然后做序列化、依赖注入、对象浏览器。C++ 呢?标准委员会讨论了二十多年,反射提案到 C++26 仍在推进,标准库至今没有 `getFields()` 这种东西。作为工程师,正确姿势不是叹息,而是看清替代拼图:`typeid` 能拿到类型身份,`type_traits` 能在编译期查询类型性质,模板特化能按类型生成定制行为,宏与代码生成工具能产出"字段表"。四块拼起来,反射的主要用途(序列化、日志、类型分派)都做得成,只是不像 Java 那样"免费"。

第二个要说清的事实:很多"需要反射"的需求其实不需要反射。想在运行时按对象类型执行不同逻辑?虚函数(第6章)就是类型分派的原生机制,效率远高于查反射表。想写"每种类型都适用的算法"?模板(第10章)在编译期就完成了分派。反射真正不可替代的场景是有限的:把对象转成字符串/JSON(序列化)、从配置/数据库按字段名填值、类型名注册表。抓住这三类场景,下文逐一给出现有工具的正解。

§11.2 `typeid` 与 `type_info`

`typeid` 是 RTTI(Run-Time Type Information,运行时类型信息)的入口:`typeid(表达式)` 或 `typeid(类型)` 返回 `std::type_info` 的引用,携带类型的身份信息。`typeid` 对多态对象给出动态类型(派生类),对非多态对象给出静态类型。`type_info` 能做的有限:`==` 判断两个类型是否相同,`hash_code()` 得到稳定哈希,`name()` 返回实现定义的名称——注意 `name()` 的格式因编译器而异(GCC 是修饰名),只适合日志,绝不能当协议字符串。

`typeid` 的实际用途比想象中窄:它适合"查类型身份"(如 `assert` 某个对象确实是预期类型)、调试与日志(打印动态类型名)。它不适合字段遍历、方法调用——`type_info` 根本不含这些信息。RTTI 的开销:每个多态类型增加一份类型信息表,`typeid` 是查表操作,代价很小;但 `-fno-rtti` 的嵌入式环境会去掉全部 RTTI,此时 `typeid` 无法编译,只能退回到模板方案。

清单 11-1 `typeid` 与 `type_info`

```

#include <iostream>
#include <string>
#include <typeinfo>

class Animal {
public:
    virtual ~Animal() = default;    // 多态:typeid 才反映动态类型
};

class Dog : public Animal {};
class Cat : public Animal {};

int main() {
    Animal* p = new Dog;
    // 多态对象:typeid 给出实际类型 Dog
    std::cout << "动态类型: " << typeid(*p).name() << "\n";
    if (typeid(*p) == typeid(Dog)) {
        std::cout << "确实是 Dog\n";
    }
    // 非多态对象:typeid 只认静态类型
    Animal plain;
    std::cout << "静态类型: " << typeid(plain).name() << "\n";

    std::cout << "hash: " << typeid(Dog).hash_code() << "\n";
    delete p;
    return 0;
}

```

C 的写法	C++ 的写法	说明
struct { int tag; union {...}; } 手写类型标记	typeid 由编译器维护	类型信息不可伪造,不会忘初始化
switch (obj->tag) 手工分派	虚函数 / dynamic_cast	C 每加类型改全部分派点,C++ 新增类零改动
tag 与真实数据可能对不上	typeid 永远与对象一致	RTTI 信息由编译器生成

§11.3 dynamic_cast:带检查的向下转型

C 的多态转型只有一个工具:(Dog*)p 裸强转——程序员保证 p 真的是 Dog,编译器不检查,错了就是未定义行为。C++ 的 dynamic_cast 把检查做进语言:dynamic_cast<Dog*>(p) 运行时核实 p 的动态类型与目标兼容,成功返回指针,失败返回 nullptr;引用版本失败抛 std::bad_cast。转型的目标必须是多态类型(含虚函数),否则编译错误——这是编译期的强制约束,杜绝了"C 里对任意结构体瞎转"的自由。

工程上 dynamic_cast 的正确位置:偶尔需要"基类指针 + 派生类专属方法"时,它比维护一整套虚函数接口要轻;但把它当常规流程(每个对象都转一遍)就是设计气味——能靠虚函数表达的多态,别靠转型表达。规则:基类接口能完成的事用虚函数;确实只有特定派生类才有的能力,用 dynamic_cast 判空后使用;判空不可省略,空指针访问是崩溃。

C 的写法	C++ 的写法	说明
(Dog*)p 裸强转,不检查	dynamic_cast<Dog*>(p) 运行时检查	C 错了是未定义行为,C++ 失败返回空
对任意结构体都能强转	目标必须多态,否则编译错误	C++ 在编译期就拦住乱转
类型判断靠手写标记字段	typeid 比较动态类型	C++ 类型信息由编译器维护

清单 11-2 dynamic_cast 安全转型

```

#include <iostream>
#include <vector>
#include <memory>

class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};

class Circle : public Shape {
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    double radius() const { return r; }          // 派生类专属方法
private:
    double r;
};

class Rect : public Shape {
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
private:
    double w, h;
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.0));
    shapes.push_back(std::make_unique<Rect>(2.0, 3.0));

    for (const auto& s : shapes) {
        if (auto c = dynamic_cast<Circle*>(s.get())) { // 失败返回空
            std::cout << "圆形,半径 " << c->radius() << "\n";
        } else {
            std::cout << "非圆形,面积 " << s->area() << "\n";
        }
    }
    return 0;
}

```

⚠ 易错点 1:dynamic_cast 的结果不判空

dynamic_cast<T*> 转型失败返回空指针,直接解引用就是空指针崩溃。引用版本失败则抛 std::bad_cast。指针版必判空,或用 if (auto x = dynamic_cast<T*>(p)) 的组合写法,把检查与使用绑在一起。

⚠ 易错点 2:对空指针做 typeid

typeid(*p) 当 p 是空指针时,对多态类型会抛出 std::bad_typeid,而不是安静地返回未知类型。日志代码最容易踩:为了打印类型名对空指针解引用。先判空再 typeid,或让空指针走独立分支。

§11.4 编译期类型查询:type_traits

反射的另一个方向是编译期:std::type_traits 头文件提供一整组类型性质查询——is_same_v 判断类型相同、is_base_of_v 判断继承关系、is_arithmetic_v 判断算术类型、is_convertible_v 判断可转换性,加上 decltype 取表达式类型,配合 if constexpr(第10章),类型判断全部在编译期完成,零运行开销。这就是 C++ 的"编译期反射":信息不藏在运行时表里,而是由模板机制在编译时推导。

类型萃取与模板特化组合,能做出"类型字典"级别的定制:为每种类型声明一个 traits 特化,提供类型名称、默认值、序列化函数等元信息;通用模板给默认实现,特化按类型覆盖。标准库就是这样组织的:std::iterator_traits、std::numeric_limits 都是 traits 的样板。这套方案的价值在"按类型定制且零开销",局限也在编译期——字段列表、运行时按名字查找字段,仍然做不了,那是 §11.5 的领域。

清单 11-3 type_traits 编译期查询

```

#include <iostream>
#include <string>
#include <type_traits>

// traits 特化:每种类型提供"类型名"
template <typename T>
struct TypeName { static constexpr const char* value = "未知"; };

template <>
struct TypeName<int> { static constexpr const char* value = "整数"; };

template <>
struct TypeName<double> { static constexpr const char* value = "浮点"; };

template <typename T>
std::string typeInfo(T v) {
    if constexpr (std::is_arithmetic_v<T>) {
        return std::string(TypeName<T>::value) + " " + std::to_string(v);
    } else {
        return std::string(TypeName<T>::value) + " " + v;
    }
}

int main() {
    std::cout << typeInfo(42) << "\n";           // 整数 42
    std::cout << typeInfo(3.14) << "\n";         // 浮点 3.14
    std::cout << typeInfo(std::string("文本")) << "\n"; // 未知 文本
    std::cout << std::boolalpha
                << std::is_base_of_v<std::string, std::string> << "\n";
    return 0;
}

```

§11.5 宏与代码生成:运行时反射的替代品

真正需要"运行时拿到字段/枚举信息"时,标准工具是宏与代码生成。X-macro 是最优雅的模式:把数据列表写一遍,用宏把它展开进多份代码——一份生成枚举定义,一份生成枚举到字符串的映射,一份生成字符串到枚举的解析。改动列表只需改一处,三份代码同步更新,彻底消灭"枚举与字符串数组对不上"这类 C 时代的经典 bug。这就是"代码生成"思想的手工版:让数据定义成为唯一事实来源。

生产环境通常直接用现成的代码生成器:protobuf 从 .proto 文件生成 C++ 序列化类,flatbuffers 同理,生成的代码自带字段表、序列化、反射接口;header 级库方面,PFR(基于模板技巧在编译期数出结构体字段个数,无宏)、magic_enum(枚举与字符串互转)、reflect-cpp(完整序列化框架)都是经过生产检验的选择。选型建议:字段多、要跨语言 → protobuf 类工具;只想要枚举转字符串 → magic_enum;只想少写样板 → PFR 或 reflect-cpp。它们共同的哲学与 X-macro 一致:元信息由工具生成,而不是手写。

C 的写法	C++ 的写法	说明
枚举与字符串数组两份手工维护	X-macro:一份列表生成多份代码	单一事实来源,改一处全同步
结构体字段表手写,易对不上	protobuf / PFR 生成字段信息	生成代码由工具保证一致
类型名映射手写数组	type_traits + 特化自动查询	编译期查询,零运行开销

```

#include <iostream>
#include <string>

// 数据列表:唯一事实来源
#define COLOR_LIST(X) \
    X(Red) \
    X(Green) \
    X(Blue) \
    X(Count)

// 第一次展开:生成枚举
enum Color {
#define X(name) name,
    COLOR_LIST(X)
#undef X
};

// 第二次展开:生成枚举到字符串的映射
const char* colorName(Color c) {
    switch (c) {
#define X(name) case name: return #name;
        COLOR_LIST(X)
#undef X
    }
    return "未知";
}

// 第三次展开:生成字符串到枚举的解析
bool colorParse(const std::string& s, Color& out) {
#define X(name) if (s == #name) { out = name; return true; }
    COLOR_LIST(X)
#undef X
    return false;
}

int main() {
    Color c = Green;
    std::cout << colorName(c) << "\n";           // Green
    Color parsed;
    if (colorParse("Blue", parsed)) {
        std::cout << "解析成功 " << colorName(parsed) << "\n";
    }
    return 0;
}

```

⚠ 易错点 3:宏不尊重作用域,展开不可调试

X-macro 的 #define 会污染全局命名空间,#undef 必须与 #define 成对,宏内的拼接与字符串化写法晦涩,调试时断点打在展开后的代码上。规则:宏模式只用于"多份代码必须同步"的机械展开;能写成模板或 constexpr 的逻辑,优先语言特性,宏是最后的工具。

§11.6 生产环境的选择:需求对表

把本章的工具按需求对表:需要对象转 JSON/序列化——protobuf、reflect-cpp 或手写 toJson(模板+if constexpr 自动遍历);需要枚举与字符串互转——magic_enum,一个头文件;需要按字段名从配置填值——代码生成器生成的解析器;需要类型名/注册表——typeid 加静态注册;需要按运行时类型分派——虚函数,而不是反射。对照下来你会发现,Java 反射解决的问题,在 C++ 里几乎都有专门的正解,代价是"每个问题都要选对工具",而不是一个万能 API。

还有一个常被忽略的工具是静态注册:程序启动时用全局对象的构造函数把"类型名与工厂函数"登记进注册表(见清单 11-5),运行时按字符串查找。配合 typeid 的类型名与模板注册函数,几十行代码就能实现插件系统与按名创建对象,这是反射需求里性价比最高的自建方案,也是 codegen 之外最常见的工程实践。

最后是心态建议:不要为"没有反射"写哀歌,也别为"假装有反射"造一个蹩脚的通用框架。C++ 的哲学是把能力分散给专门的机制:多态给虚函数,编译期计算给模板,元信息给生成代码。三者的组合比单一反射更强——虚函数快、模板零开销、生成代码可审计,这正是 C++ 在系统级软件里不可替代的原因。

清单 11-5 类型注册表:typeid 哈希做键

```
#include <functional>
#include <map>
#include <memory>
#include <typeinfo>
#include <string>

struct Registry {
    // 类型名 -> 工厂函数
    std::map<std::string, std::function<void*>> makers;

    template <typename T>
    void reg() {
        // 模板注册:自动取类型名
        makers[typeid(T).name()] = [] { return new T(); };
    }

    void* create(const std::string& name) {
        auto it = makers.find(name);
        return it == makers.end() ? nullptr : it->second();
    }
};

struct A {}; struct B {};

int main() {
    Registry r;
    r.reg<A>();
    r.reg<B>();
    void* a = r.create(typeid(A).name()); // 按名字创建
    std::cout << (a ? "A 创建成功" : "A 创建失败") << "\n";
    delete static_cast<A*>(a);
    return 0;
}
```


反射与动态特性速查表

概念	语法示例	要点
RTTI	运行时类型信息	typeid 与 dynamic_cast 的基础
typeid	typeid(x) / typeid(T)	多态对象返回动态类型
type_info	typeid(x).name()	name 实现定义,只适合日志
类型比较	typeid(a) == typeid(B)	判断动态类型是否相同
hash_code	typeid(T).hash_code()	稳定哈希,可做注册表键
dynamic_cast	dynamic_cast<T*>(p)	多态向下转型,失败返回空
bad_cast	引用版转型失败抛出	catch 处理引用转型失败
类型萃取	std::is_same_v<T, U>	编译期查询,零运行开销
继承判断	std::is_base_of_v<B, D>	编译期判断继承关系
decltype	decltype(expr)	取表达式类型
X-macro	列表宏展开多份代码	枚举/字符串/解析同步生成
magic_enum	magic_enum::enum_name(v)	枚举字符串互转,单头文件
PFR	pfr::get<i>(obj)	编译期结构体字段访问
代码生成	protobuf / flatbuffers	从描述文件生成序列化代码

自测题

- Q 1. C++ 为什么没有 Java 那样的运行时反射?标准反射提案的现状如何?**
- Q 2. typeid 返回什么?type_info::name() 为什么不能当协议字符串?**
- Q 3. dynamic_cast 与 C 的裸强转有何区别?为什么目标类型必须多态?**
- Q 4. type_traits 能实现哪类"反射"?它有什么局限?**
- Q 5. X-macro 为什么能保证枚举与字符串数组一致?生产环境有哪些现成替代?**

✓ 参考答案

- 反射提案在标准委员会讨论了二十多年,C++26 仍在推进,标准库没有 getFields 之类的接口。设计上 C++ 优先零开销抽象与显式机制,反射的通用性以运行时开销与代码膨胀为代价,与语言哲学相悖。
- typeid 返回 std::type_info 引用,描述类型的身份;name() 的格式由编译器实现决定(GCC 是修饰名),不同编译器不同版本都可能不同,只适合日志展示,作为协议字符串会跨编译器失配。
- C 的裸强转不做任何检查,类型错了是未定义行为;dynamic_cast 运行时核实动态类型,失败指针版返回空、引用版抛 bad_cast。目标必须多态是因为检查要查虚表里的类型信息,非多态类没有虚表。
- type_traits 在编译期回答类型性质问题:是否相同、是否算术类型、是否继承、是否可转换,配合 if constexpr 做零开销分派。局限:没有运行时字段表,不能按名字查字段、不能枚举方法。
- X-macro 只有一份数据列表,枚举定义、名称映射、解析函数都从它展开,改列表即改全部,不存在两份手工维护的偏差。生产替代:magic_enum(枚举)、PFR(结构体字段)、protobuf/reflect-cpp(完整序列化)。

实验:日志系统的类型感知输出

本实验构建一个带类型感知的日志与序列化系统:typeid 打印动态类型,type_traits 与 if constexpr 按类型格式化数值与文本,traits 特化提供类型名,X-macro 定义日志级别枚举与字符串映射,dynamic_cast 安全访问派生类专属字段。代码分三部分,覆盖点已在注释中标明。

实验 11•第一部分:类型名与格式化(覆盖:typeid、type_traits、if constexpr、traits 特化)

```
#include <iostream>
#include <string>
#include <typeinfo>
#include <type_traits>
#include <sstream>

// 覆盖:traits 特化—每个类型提供中文类型名
template <typename T>
struct TypeName { static constexpr const char* value = "未知"; };
template <>
struct TypeName<int> { static constexpr const char* value = "整数"; };
template <>
struct TypeName<double> { static constexpr const char* value = "浮点"; };
template <>
struct TypeName<std::string> { static constexpr const char* value = "文本"; };

// 覆盖:type_traits + if constexpr 按类型格式化
template <typename T>
std::string formatValue(const T& v) {
    if constexpr (std::is_arithmetic_v<T>) {
        return std::to_string(v);
    } else if constexpr (std::is_same_v<T, std::string>) {
        return v;
    } else {
        return "?";
    }
}

// 覆盖:typeid 打印动态类型名,type_info 比较
std::string dynamicTag(const void* p, const std::type_info& dyn) {
    (void)p;
    return dyn.name();           // 实现定义的名称,仅日志用
}
```

实验 11•第二部分:日志级别与动态转型(覆盖:X-macro、dynamic_cast、多态)

```

// 覆盖:X-macro—日志级别:一份列表,生成枚举与字符串
#define LEVEL_LIST(X) \
    X(Debug) \
    X(Info) \
    X(Warn) \
    X(Error) \
    X(Count)

enum Level {
#define X(name) name,
    LEVEL_LIST(X)
#undef X
};

const char* levelName(Level l) {
    switch (l) {
#define X(name) case name: return #name;
        LEVEL_LIST(X)
#undef X
    }
    return "?";
}

// 覆盖:多态基类,为 dynamic_cast 提供目标
struct LogEntry {
    virtual ~LogEntry() = default;
    virtual std::string summary() const = 0;
};

struct ErrorEntry : LogEntry {          // 派生类专属字段
    int code;
    explicit ErrorEntry(int c) : code(c) {}
    std::string summary() const override {
        return "错误码 " + std::to_string(code);
    }
};

struct TextEntry : LogEntry {
    std::string text;
    explicit TextEntry(std::string t) : text(std::move(t)) {}
    std::string summary() const override { return text; }
};

```

实验 11•第三部分:main 调度(覆盖:typeid 分派、dynamic_cast 判空、综合输出)

```

int main() {
    // 覆盖:typeid 动态类型识别
    TextEntry t("连接建立");
    LogEntry* p = &t;
    std::cout << "动态类型: " << typeid(*p).name() << "\n";
    if (typeid(*p) == typeid(TextEntry)) {
        std::cout << "识别为文本条目\n";
    }

    // 覆盖:dynamic_cast 判空后访问派生类专属字段
    LogEntry* e = new ErrorEntry(500);
    if (auto err = dynamic_cast<ErrorEntry*>(e)) {
        std::cout << "错误码 " << err->code << "\n";
    } else {
        std::cout << "不是错误条目\n";
    }
    delete e;

    // 覆盖:traits 类型名 + 格式化 + X-macro 级别
    int n = 42;
    std::cout << levelName(Warn) << " " << TypeName<decltype(n)>::value
        << " " << formatValue(n) << "\n";
    std::cout << levelName(Info) << " " << TypeName<std::string>::value
        << " " << formatValue(std::string("运行中")) << "\n";
    std::cout << levelName(Error) << " " << TypeName<double>::value
        << " " << formatValue(0.5) << "\n";
    return 0;
}

```

实验覆盖点核对:typeid 动态类型识别与 type_info 比较、dynamic_cast 判空后访问派生类专属字段、type_traits 的 is_arithmetic_v/is_same_v 与 if constexpr 格式化、traits 特化提供类型名(TypeName)、X-macro 生成枚举与字符串映射(levelName)、多态基类与虚函数(summary)。扩展练习:给 Level 增加解析函数(字符串转枚举);或实现一个简单的 Entry 注册表,用 typeid(T).hash_code() 做键。

工程建议 1:日志与序列化优先生成代码

新项目要序列化与日志反射,第一时间评估 protobuf、flatbuffers、reflect-cpp,而不是手写 X-macro 与映射函数:生成代码经得起生产考验,字段变更全自动同步,还有跨语言红利。手写元信息只适合字段极少、库引入成本高的场合。

工程建议 2:分派优先虚函数,转型是最后手段

代码里出现"逐个 dynamic_cast 试类型"是设计信号:把分支逻辑上移为虚函数或访问者模式,新类型不用改分发代码。dynamic_cast 保留给"偶尔需要派生类专属能力"的少数场景,并且判空永远不省。

本章小结

C++ 没有 Java 式反射,但四块工具各司其职:typeid 识别类型身份,dynamic_cast 做安全检查和转型,type_traits 在编译期查询类型,宏与代码生成提供运行时元信息。序列化、日志、类型分派的真实解法都落在这四块上。下一章进入注解与编译期指令:标准属性。